



华中科技大学

第一章 计算机系统概述



主要内容



华中科技大学

- ◆ 1.1 计算机的发展历程
- ◆ 1.2 计算机系统的基本组成和功能
- ◆ 1.3 软件开发和执行过程概述
- ◆ 1.4 计算机系统的层次结构
- ◆ 1.5 计算机系统性能评价



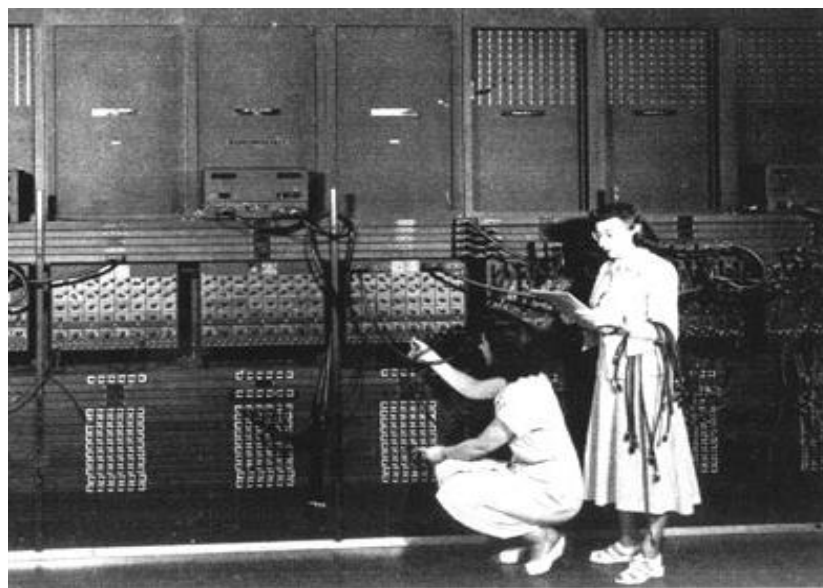
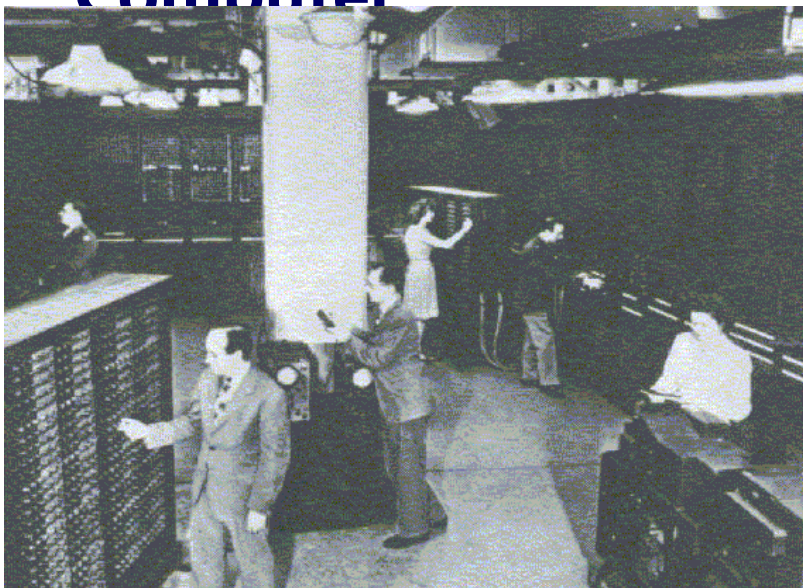
诞生



华中科技大学

1946, 美国, ENIAC

Electronic Numerical Integrator And Computer



第一代



华中科技大学

20世纪40年代中期——50年代末

逻辑元件：电子管

存储器：声延迟线或磁鼓

定点运算

没有软件，使用低级语言编程

典型机器：ENIAC、EDVAC、EDSAC

IAS计算机：“存储程序”



第二代



华中科技大学

20世纪50年代中后期——60年代中

逻辑元件：晶体管

内存：磁芯存储器

外存：磁鼓、磁带

浮点运算、变址、中断、I/O处理器

高级语言和编译器

**典型机器：TRADIC、TX-2、IBM 7070、IBM
7090、IBM 1401**



第三代



华中科技大学

60年代中期——70年代中后期

集成电路

微程序控制器、高速缓存、虚拟存储器、流水线

操作系统

典型机器：IBM 360、CDC 6600、CDC 7600、

CYBER

CDC STAR-100、ILLIAC IV

DEC PDP



第四代



华中科技大学

70年代后期至今

超大规模集成电路

计算机网络

普适计算

嵌入式

智能化

典型机器：Cray-1

PC



超大规模集成电路

表 1.1 Intel CPU 芯片性能比较

芯片名	年代	集成度 (晶体管)	主频范围 (MHz)	线宽 (μm)	数据总线宽 (位)	地址总线宽 (位)
4004	1971	2300	0.74	2	4	
8080	1974	8000	4	1.5	8	16
8086	1978	2.9 万	4.77	1.5	16	20
80286	1984	13.5 万	6 ~ 25	1 ~ 1.5	16	24
80386	1985	27.5 万	20 ~ 33	1 ~ 1.5	32	32
80486	1989	118.5 万	33 ~ 100	1	32	32
Pentium	1993	300 万	60 ~ 133	0.6	64	32
Pentium Pro(P6)	1995	550 万	150 ~ 233	0.6	64	32
PII(P6 + MMX)	1997	750 万	233 ~ 400	0.35	64	32
PIII	1999	950 万	450 ~ 1000	0.25	64	32
P4	2000	4200 万	1500 ~ 3800	0.18	64	32
Core i7	2010	117 000 万	3333	0.032	64	64

主要内容



华中科技大学

- ◆ 1.1 计算机的发展历程
- ◆ 1.2 计算机系统的基本组成和功能
- ◆ 1.3 软件开发和执行过程概述
- ◆ 1.4 计算机系统的层次结构
- ◆ 1.5 计算机系统性能评价



1.2 计算机系统的基本组成和功能



华中科技大学

计算机是一种能**自动对数字化**信息进行
算术和逻辑运算的高速处理装置。





華中科技大學

一、基本功能

- 数据处理
- 数据存储
- 数据传送





一、基本功能

数据处理

- ◆ 加、减、乘、除
- ◆ 与、或、非
- ◆ 处理对象：数值、文字、图形、图像、声音、视频



一、基本功能

数据存储

- ◆ 程序
- ◆ 数据
- ◆ 存储部件：非易失存储器和快速存储器



二、硬件

1. 冯.诺依曼结构

- “存储程序”
- 运算器、控制器、存储器、输入设备、输出设备
- 存储数据和指令；自动执行；加减乘除与或非；
人机通过输入输出设备
- 二进制表示指令和数据；指令包括操作码和地址
码；一串指令组成程序

二、硬件



华中科技大学

2. 硬件组成

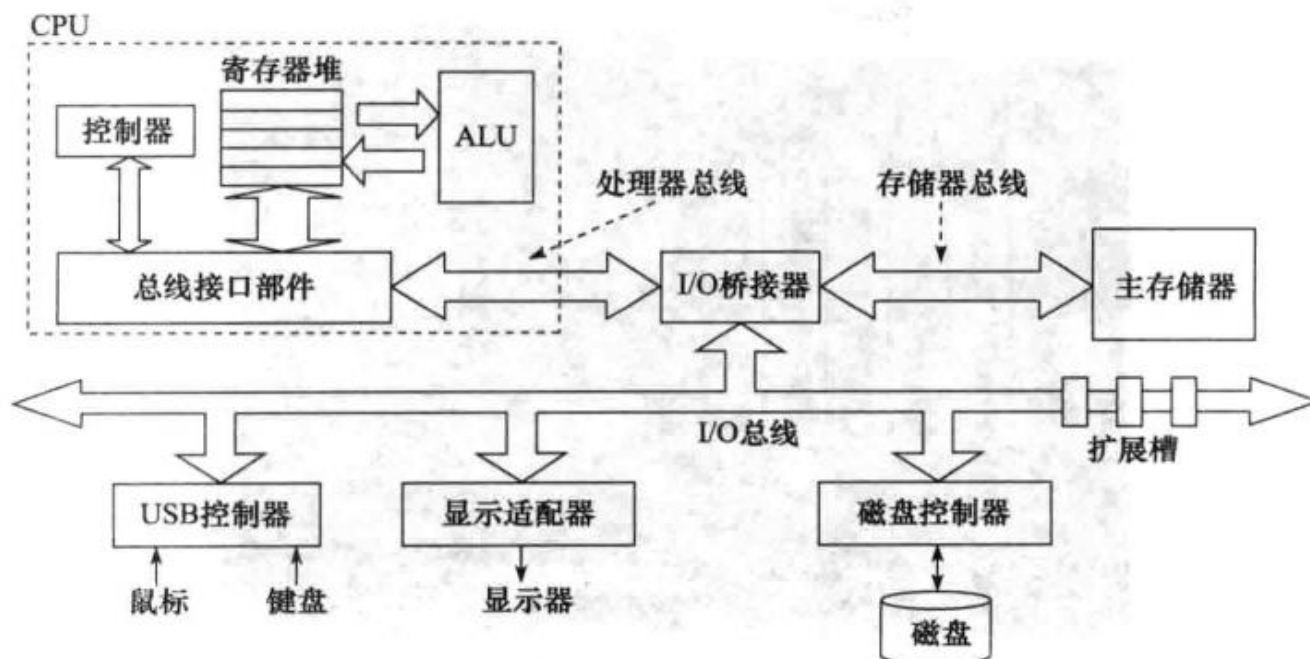


图 1.1 一个典型计算机系统的硬件组成

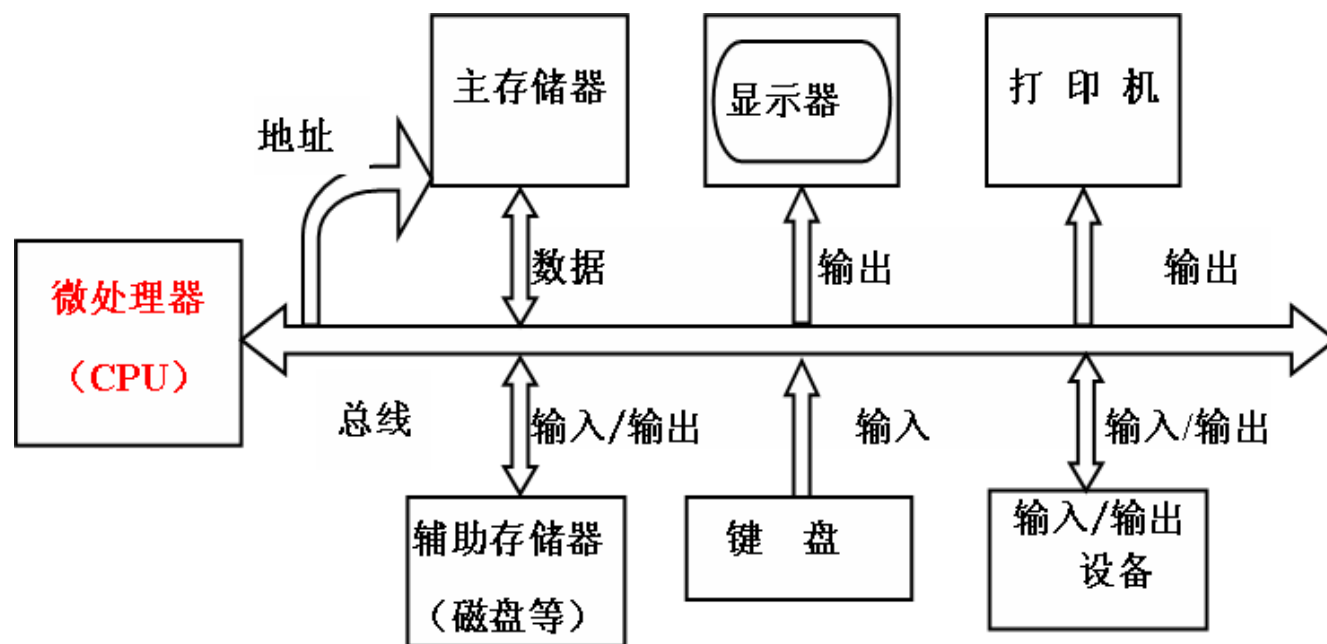


二、硬件



华中科技大学

简化的硬件组成





2. 硬件组成

- CPU
- 主存储器
- 总线
- 外部设备
 - 磁盘
 - 显示器
 - 键盘
 - 鼠标

3. CPU 结构



华中科技大学

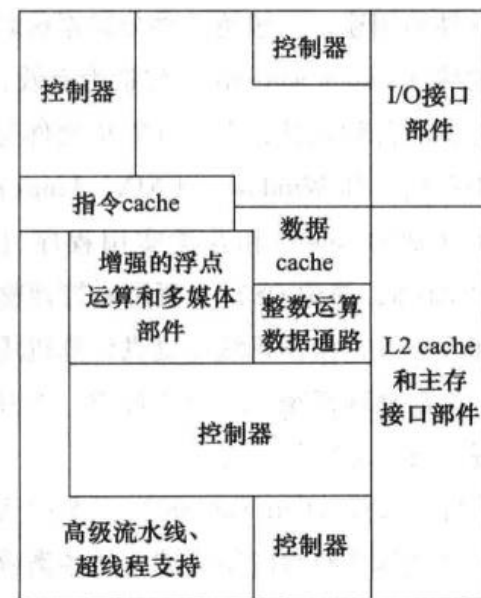
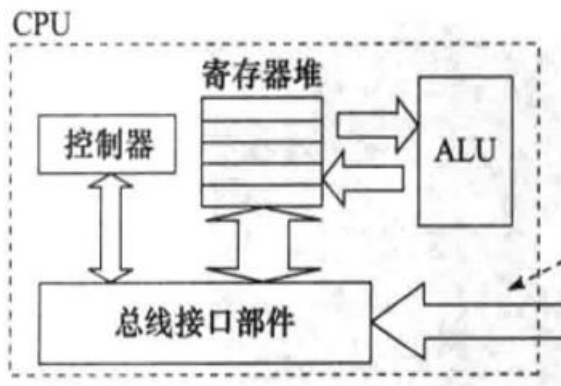


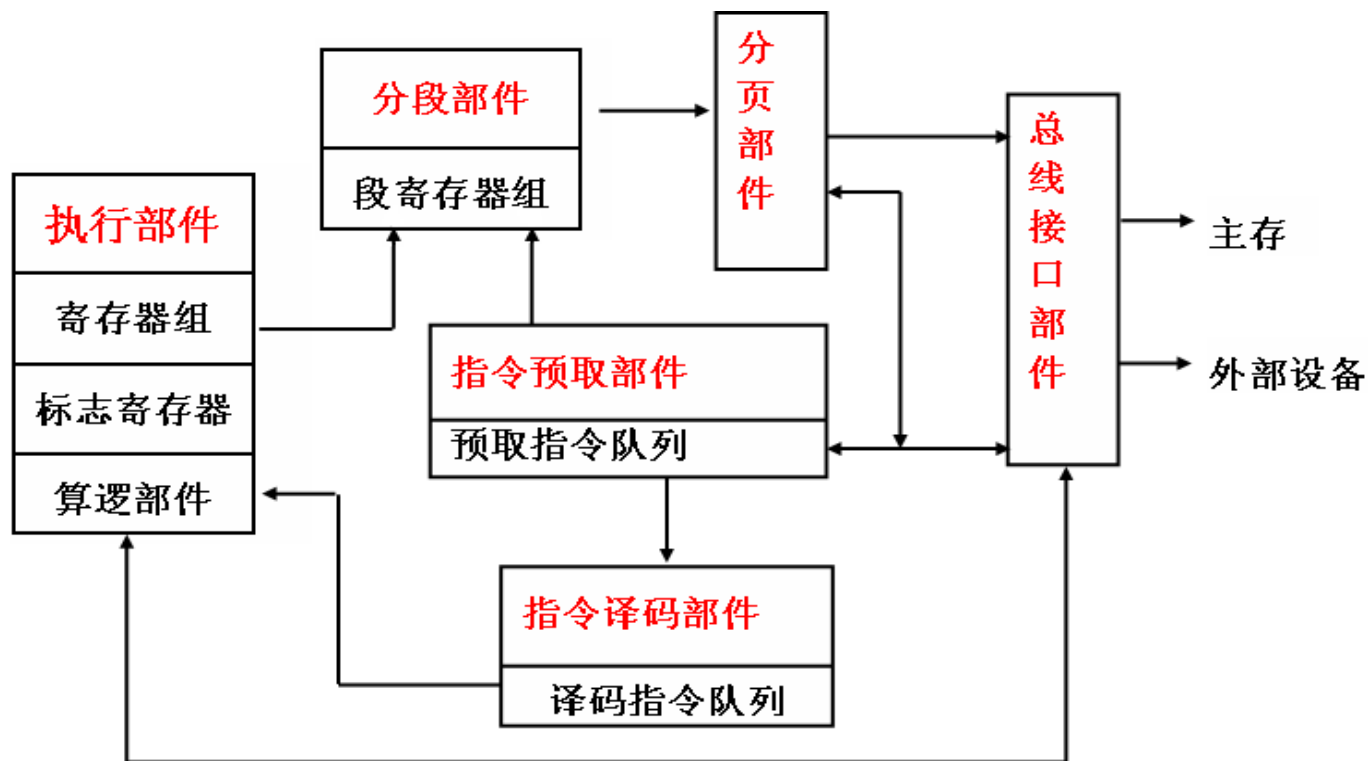
图 1.5 奔腾4 处理器芯片的内部组成



3. CPU 结构



华中科技大学



3. CPU 结构



华中科技大学

(1) 总线接口部件: 是CPU与整个计算机系统之间的高速接口

功能: 接受所有的总线操作请求, 并按优先权进行选择, 最大限度地利用本身的资源为这些请求服务。

(2) 执行部件: 寄存器组、标志寄存器、算逻部件、控制部件等组成

功能: 从译码指令队列中取出指令并且执行



寄存器组



华中科技大学

	31	16	15	8	7	0	
EAX					AH (AX)	AL	累加器
EBX					BH (BX)	BL	基址寄存器
ECX					CH (CX)	CL	计数寄存器
EDX					DH (DX)	DL	数据寄存器
ESP					SP		栈指针寄存器
EBP					BP		基址指针寄存器
ESI					SI		源变址寄存器
EDI					DI		目标变址寄存器
EIP					IP		指令指针寄存器
EFLAGS					FLAGS		标志寄存器
					CS		代码段
					SS		堆栈段
					DS		数据段
					ES		附加段
					FS		附加段
					GS		附加段

图 3.2 IA-32 的定点寄存器组



(a) 数据寄存器组(EAX, EBX, ECX, EDX)

- EAX (累加器)Accumulator
- EBX (基址寄存器)Base
- ECX (计数寄存器)Count
- EDX (数据寄存器)Data

作用：用来保存操作数、运算结果或作指示器、变址寄存器，减少存取操作数所需要的访问总线和主存储器的时间，加快运行速度。

(b) 指示器变址寄存器组(ESI、EDI、ESP、EBP)

作用：一般用来存放操作数的偏移地址，用作指示器或变址寄存器。

- ESI(Source Index)，称为源变址寄存器；字符串指令源操作数的指示器
- EDI(Destination Index)，称为目的变址寄存器；字符串指令目的操作数的指示器
- ESP (Stack Pointer) ，称为堆栈指示器，存放的是当前堆栈段中栈顶的偏移地址
- EBP(Base Pointer)，为对堆栈操作的基址寄存器；

(c) 标志寄存器

保存一条指令执行后，CPU所处状态的信息。

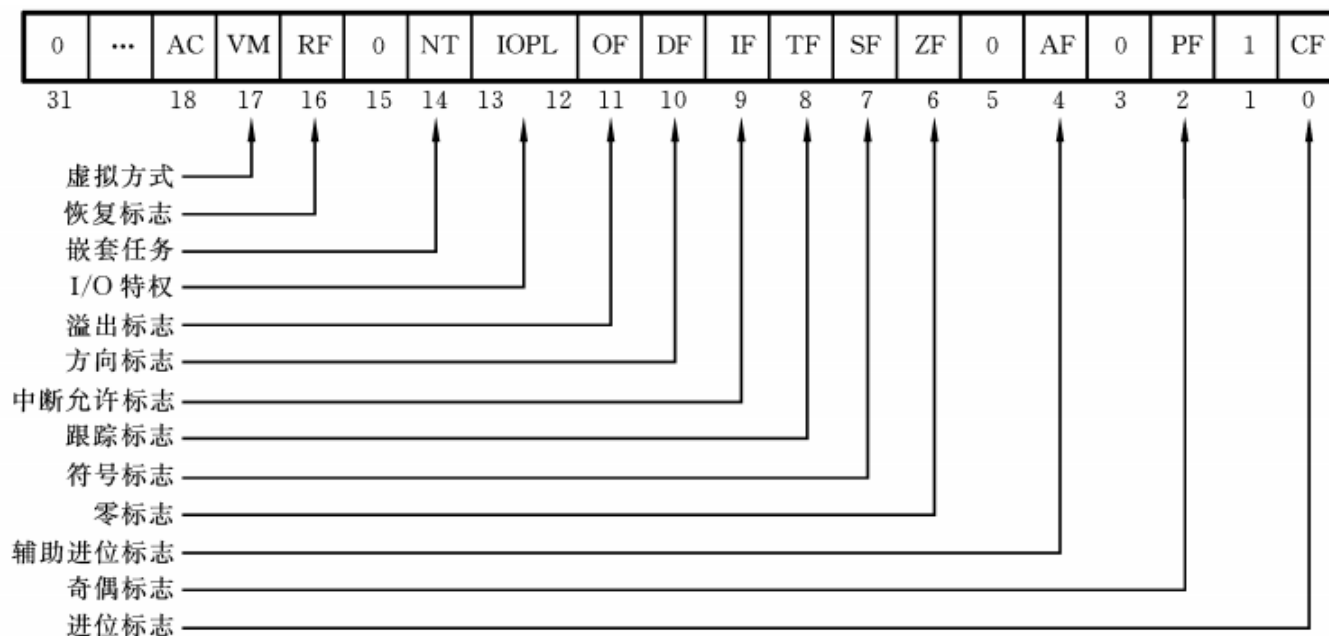


图 1.21 80X86 的标志寄存器

条件标志

- OF: 溢出标志
- SF: 符号标志
- ZF: 零标志
- CF: 进位标志

控制标志

- DF: 方向标志
- IF: 中断允许标志
- TF: 陷阱标志 (单步执行)

3. CPU 结构



华中科技大学

(3) 指令预取部件和指令译码部件

指令预取部件:通过总线接口部件, 把将要执行的指令从主存中取出, 送入指令排队机构中排队。

指令译码部件:从指令预取部件中读出指令并译码, 再送入译码指令队列排队供执行部件使用。

指令指示器EIP:它总是保存着下一条将要被CPU执行的指令的偏移地址, 其值为该指令到所在段首址的字节距离。



3. CPU 结构



华中科技大学

(4) 分段部件和分页部件

(a) 使用分段部件和分页部件实现虚拟存储空间映射到物理存储空间

程序员使用二维地址 “段地址：段内偏移地址”

分段部件

段地址：段内偏移地址 -----> 一维的线性的地址

分页部件

虚拟的线性的地址 -----> 存储器的物理地址





分段部件和分页部件

(b) 段寄存器：保存段首描述符

- 代码段寄存器 CS
- 堆栈段寄存器 SS
- 数据段寄存器 DS
- 附加段寄存器 ES
- 附加段寄存器 FS
- 附加段寄存器 GS

三、软件



华中科技大学

- 系统软件

- 操作系统
- 语言处理系统
- 系统工具

- 应用软件

- 数据处理
- 科学计算
- 事务处理
- 多媒体处理
- 工程设计
- 过程控制等



主要内容



华中科技大学

- ◆ 1.1 计算机的发展历程
- ◆ 1.2 计算机系统的基本组成和功能
- ◆ 1.3 软件开发和执行过程概述
- ◆ 1.4 计算机系统的层次结构
- ◆ 1.5 计算机系统性能评价



1.3 程序开发和执行过程概述

一、从源程序到执行程序

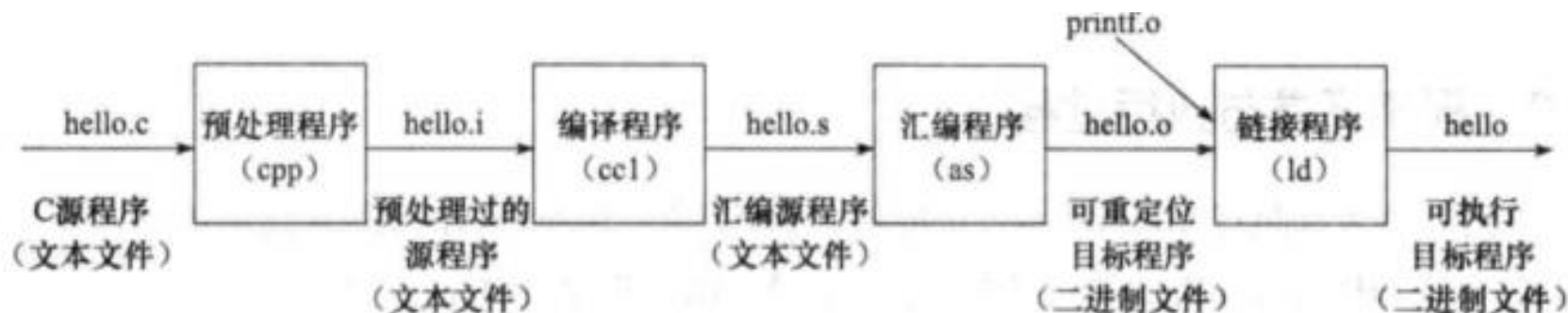


图 1.7 `hello.c` 源程序文件到可执行目标文件的转换过程



一、从源程序到执行程序

- **源程序：文本文件，用户编写。**
*.c, *.cpp, *.asm
- **目标程序：二进制文件，机器可识别，但不能执行**
.o (.obj)
- **可执行程序：二进制文件，机器可执行**
* (*.exe)
- **编译器：将源程序转为目标程序**
- **链接器：将一个或多个目标程序链接生成可执行程序**

二、可执行程序的启动和执行

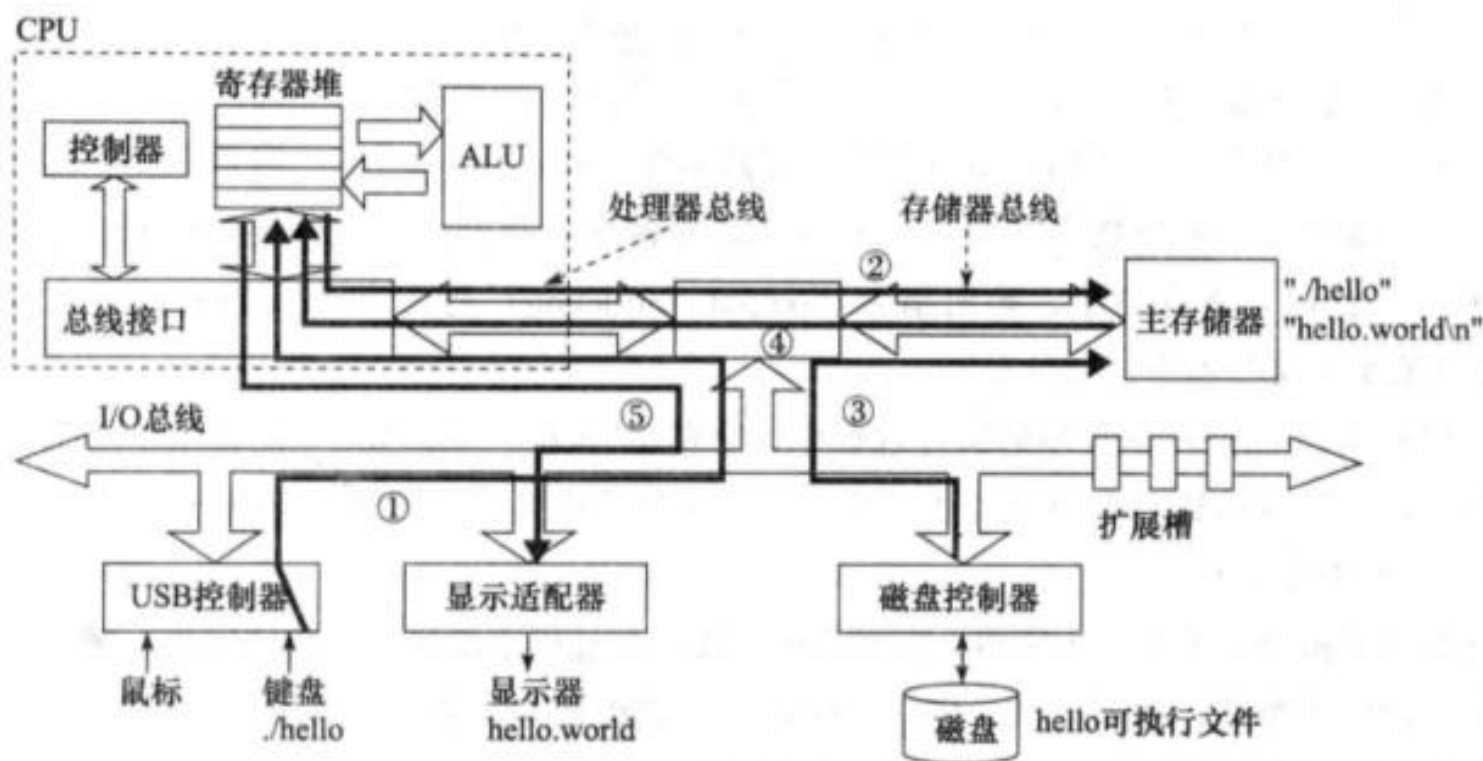


图 1.8 启动和执行 hello 程序的全过程



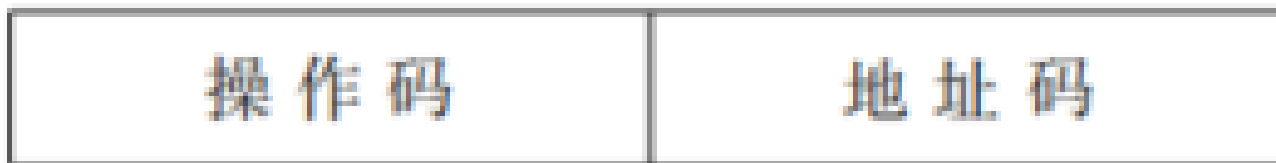
二、可执行程序的启动和执行

- 双击图标，或在命令行中输入可执行程序文件名
- 从硬盘上将可执行程序的文件加载到内存中
- 将CPU中的指令指示寄存器EIP，指向程序的第一条指令
- CPU开始执行程序，按程序执行每条指令



三、程序中每条指令的执行

机器指令的一般形式为





三、程序中每条指令的执行

机器指令完成某一个基本操作。例如：

- 数值的传送
- 加、减、乘、除
- 与、或、非
- 跳转
- 输入/输出



三、程序中每条指令的执行

每条指令的执行过程：

- 从内存中读取该指令
- 对指令进行译码
- 若为内存操作数，从内存中取操作数
- 对操作数进行运算
- 保存运算结果。若为内存操作数，保存结果到内存中



四、一个汇编语言程序的例子

1) 汇编语言——符号化的机器语言

.386

;堆栈段

STACK SEGMENT USE16 STACK ;段名和组合类型

DB 200 DUP(0) ; 堆栈的大小为200个字节

STACK ENDS

;数据段

DATA SEGMENT USE16 ;段为16位段

SUM DW ? ;SUM为字变量, 初值不定

DATA ENDS

;代码段

CODE SEGMENT USE16

ASSUME CS:CODE, SS:STACK,

DS:DATA, ES:DATA

START: MOV AX, DATA

MOV DS, AX;数据段首址送DS

MOV CX, 50;循环计数器置初值

MOV AX, 0 ;累加器置初值

MOV BX, 1 ;1→BX

NEXT:ADD AX, BX;(AX)+(BX)→AX

INC BX

INC BX ;(BX)+2→BX

DEC CX ;(CX)-1→CX

JNE NEXT ;(CX)≠0转NEXT

MOV SUM, AX ;(CX)=0累加结果→SUM

MOV AH, 4CH

INT 21H ;返回DOS

CODE ENDS

END START ;源程序结束语句。程序运行时

, 启动地址为START





四、一个汇编语言程序的例子

2) 在该例中，一共定义了三个段：**堆栈段**、**数据段**和**代码段**。

.386

;**堆栈段**

STACK SEGMENT USE16 STACK

...

STACK ENDS

;**数据段**

DATA SEGMENT USE16

...

DATA ENDS

;**代码段**

CODE SEGMENT USE16

 ASSUME CS:CODE, SS:STACK,

 DS:DATA, ES:DATA

START: MOV AX, DATA

 MOV DS, AX

...

 MOV AH, 4CH

 INT 21H

CODE ENDS

 END START



四、一个汇编语言程序的例子

3) 汇编语句的一般格式:

[名字] 操作符 [操作数或地址] [;注释]

a) 操作符可分为3类

- 指令
- 伪指令
- 宏

b) 操作数也可分为3类

- 数值操作数
- 寄存器操作数
- 内存操作数

c) 常用指令:

MOV AX, 0
ADD AX, BX
INC AX
DEC CX
等





四、一个汇编语言程序的例子

4) 汇编过程

a) 必备软件

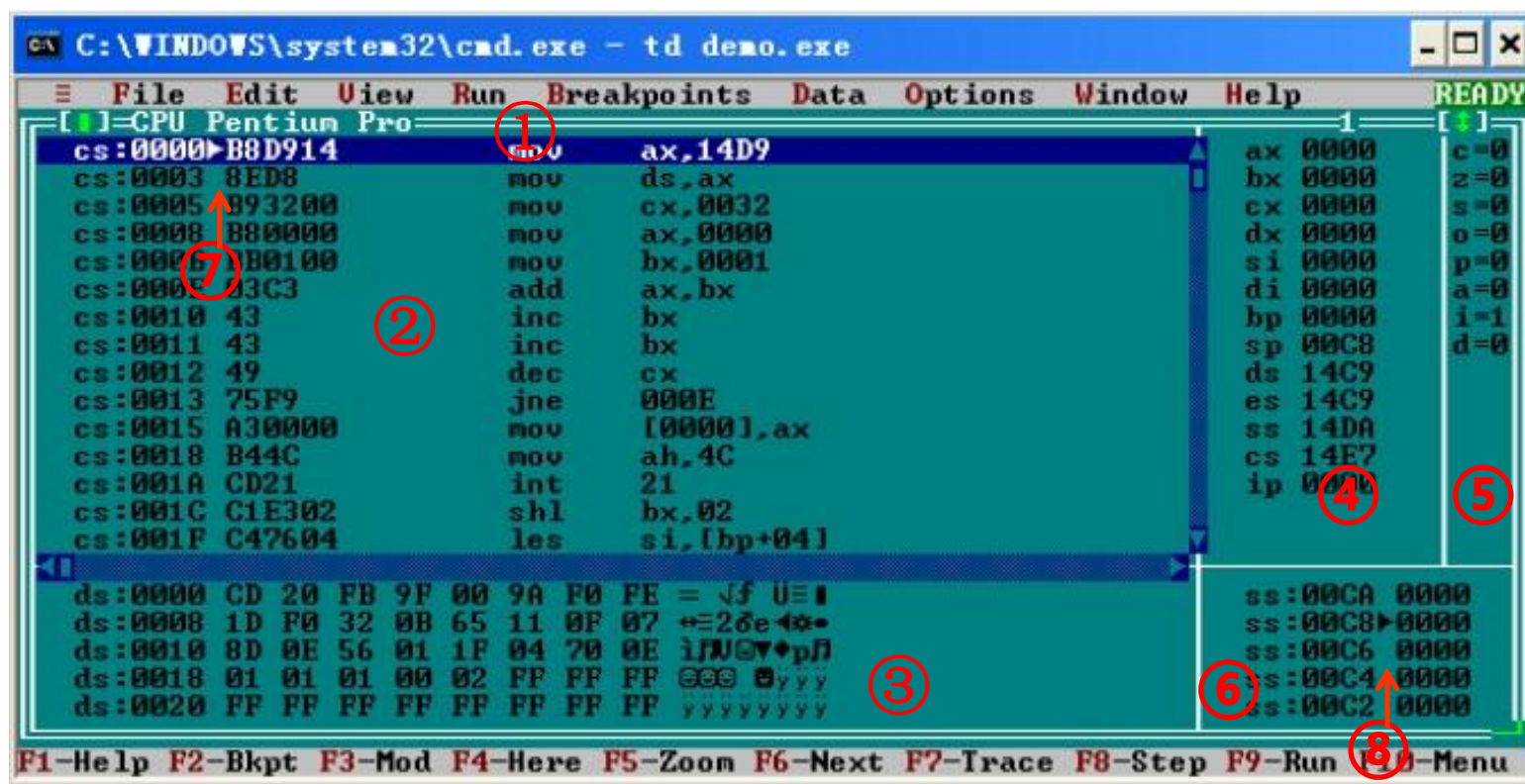
- MASM.EXE
- LINK.EXE
- TD.EXE
- 其他：文本编辑器
DOSBOX等

b) 命令

- MASM DEMO.ASM'
汇编DEMO.ASM, 应该生成DEMO.OBJ
- LINK DEMO.OBJ
链接DEMO.OBJ, 应该生成DEMO.EXE
- TD DEMO.EXE
调试DEMO.EXE

四、一个汇编语言程序的例子

5) 调试程序Turbo Debugger的使用





四、一个汇编语言程序的例子

5) 调试程序Turbo Debugger的使用

a) TD的界面分为

① 菜单

② 代码操作区

③ 数据操作区

④ 寄存器操作区

⑤ 标记寄存器操作区

⑥ 堆栈段操作区

界面中的标记

⑦ 当前指令标记

⑧ 当前栈顶标记



四、一个汇编语言程序的例子

5) 调试程序Turbo Debugger的使用

b) TD的主要操作

- 单步执行：F7、F8
- 设置（取消断点）：F2
- 直接执行：F9

c) 查看数据区

d) 查看用户界面

e) 修改数据、指令、寄存器内容

主要内容



华中科技大学

- ◆ 1.1 计算机的发展历程
- ◆ 1.2 计算机系统的基本组成和功能
- ◆ 1.3 软件开发和执行过程概述
- ◆ 1.4 计算机系统的层次结构
- ◆ 1.5 计算机系统性能评价





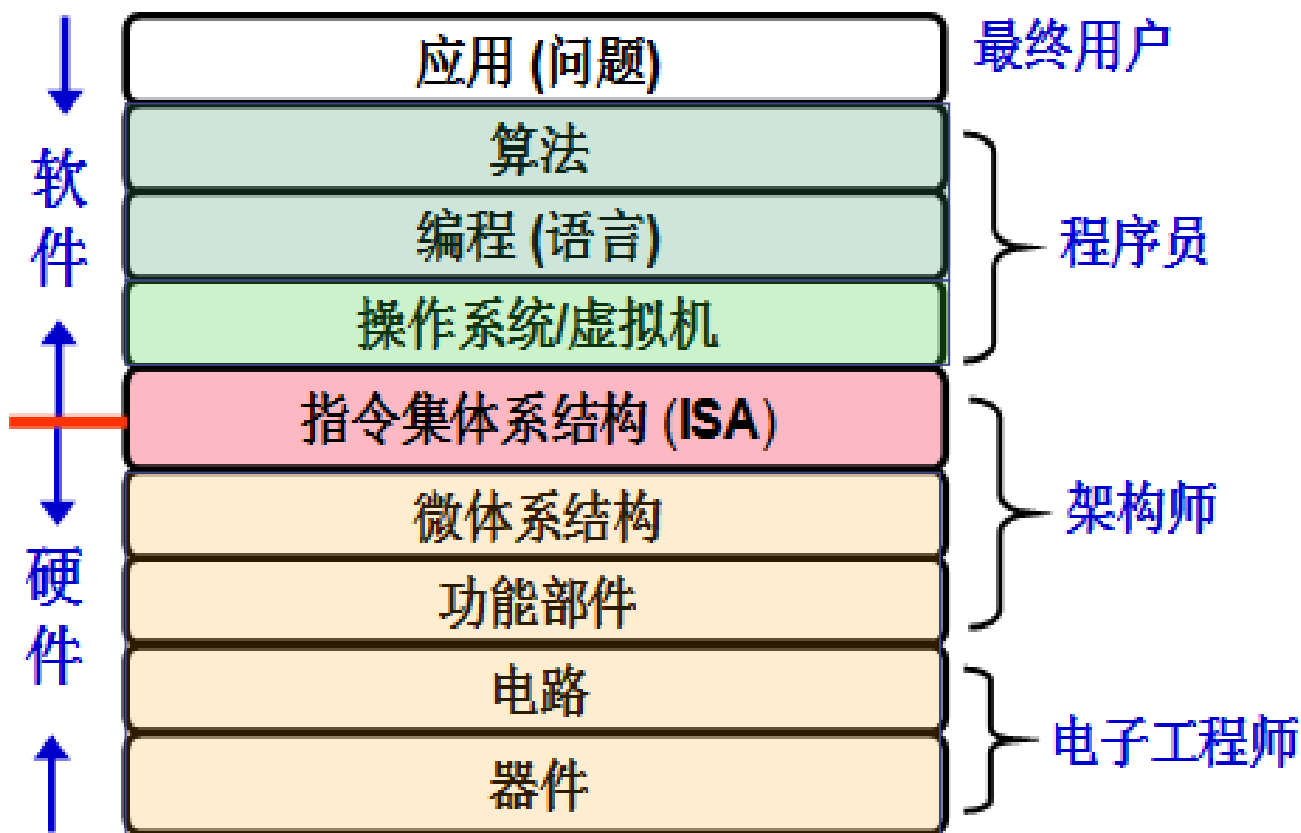
1.4 计算机系统的层次结构

一、计算机抽象层次的转换

- 计算机系统是一个层次化结构的系统
- 上层提供抽象接口
- 下层实现细节
- 计算机系统解决问题的过程，就是不同抽象层进行转换的过程



一、计算机抽象层次的转换





一、计算机抽象层次的转换

- 应用问题
- 算法
- 编程语言
 - 高级语言
 - 低级语言
 - ◆ 机器语言
 - ◆ 汇编语言



一、计算机抽象层次的转换

- 操作系统
- 指令集体系结构 (ISA)

ISA是硬件和软件的交界，是计算机系统的核心部分，ISA的内容包括：

- 指令集
- 操作数
- I/O
- 中断
- 机器状态切换
- 存储保护等



一、计算机抽象层次的转换

- 微体系结构

- 相同的ISA, 可以由不同的微体系结构实现。例如:

- Intel X86是ISA

- Intel的实现: i386, i486, i586, ...

- AMD的实现: K5, K6, K7, ..., A6, A8, A10, ...

- 微体系结构由功能部件组成

- 功能部件

- 功能部件由逻辑电路实现

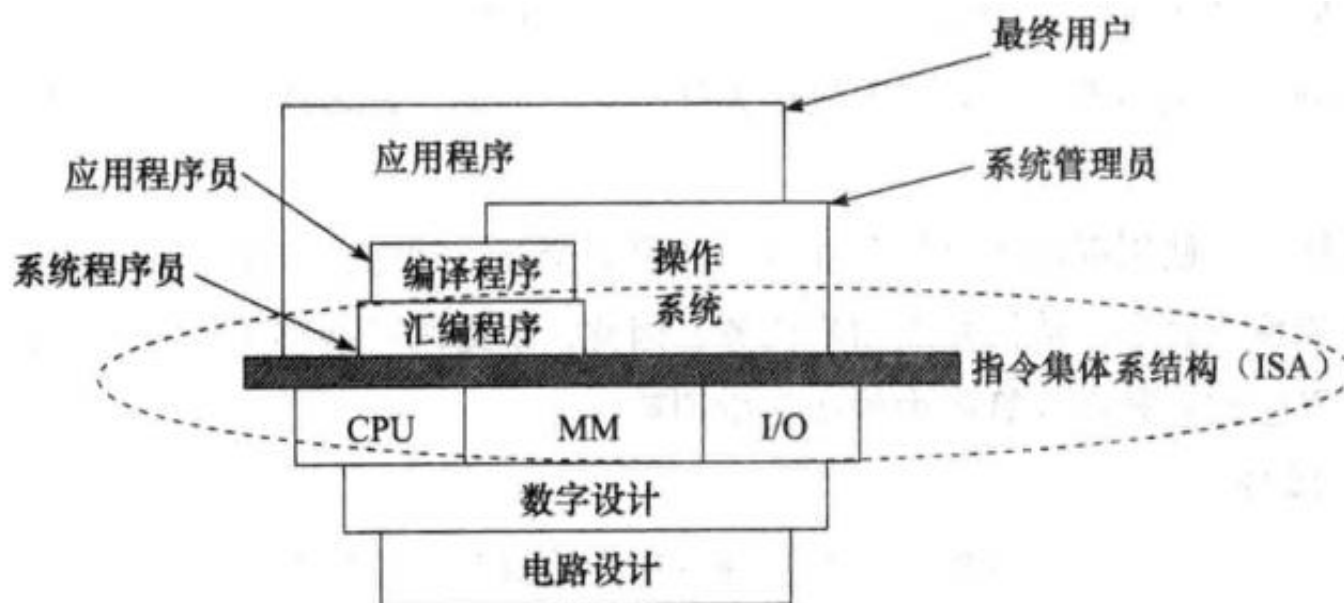
- 电路

- 器件

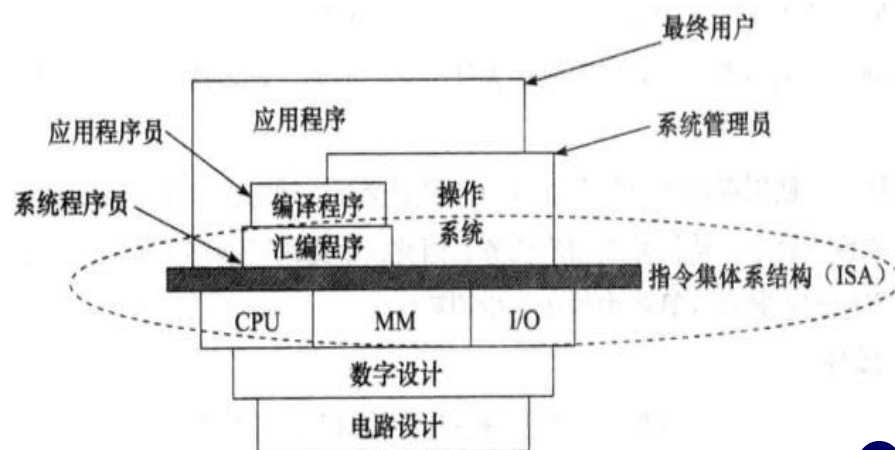




二、计算机系统的不同用户



二、计算机系统的不同用户



- 最终用户
- 系统管理员
- 应用程序员
- 系统程序员



二、计算机系统的不同用户

ISA是硬件和软件的交界，是计算机系统的核心部分

- **ISA**下面是硬件部分
- **ISA**上面是软件部分
- 硬件所有功能由**ISA**体现
- 程序员看到的机器属性是**ISA**层面的内容，看到的机器是配置了指令系统的机器。

主要内容



华中科技大学

- ◆ 1.1 计算机的发展历程
- ◆ 1.2 计算机系统的基本组成和功能
- ◆ 1.3 软件开发和执行过程概述
- ◆ 1.4 计算机系统的层次结构
- ◆ 1.5 计算机系统性能评价





华中科技大学

一、计算机系统性能定义

- 吞吐率
- 响应时间
 - ◆ 执行时间
 - ◆ 等待时间





二、计算机系统性能测试

1. 用程序的执行时间测试计算机性能

- **CPU时间**

- ◆ 用户**CPU**时间

- ◆ 系统**CPU**时间

- **其他时间**

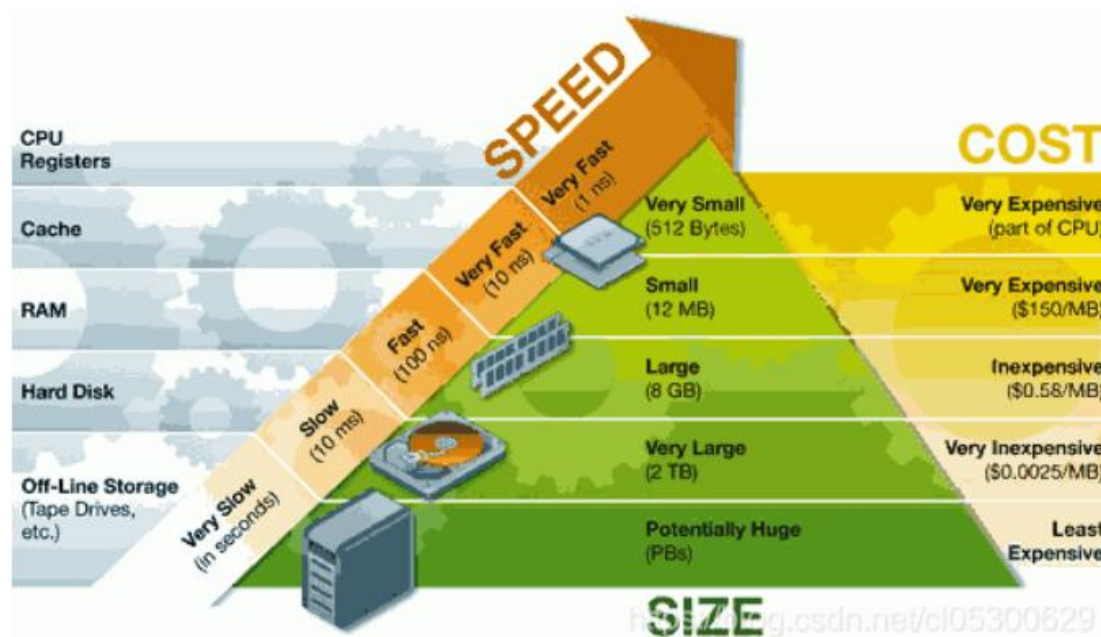
- ◆ 磁盘访问时间

- ◆ 存储器访问时间

- ◆ 输入输出时间

- ◆ 操作系统额外开销

二、计算机系统性能测试



CPU的时间观:

<https://blog.csdn.net/cl05300629/article/details/92798887>



二、计算机系统性能测试

2. **CPU性能**和系统性能不等价，主要考虑**CPU性能**，即用户**CPU时间**。

- 时钟周期/时钟频率
- **CPI**（指令所需时钟周期数）
 - 一条指令**CPI**
 - 平均**CPI**



二、计算机系统性能测试

- 1) 用户**CPU**时间 = 程序总时钟周期数 * 时钟周期
(或 = 程序总时钟周期数 / 时钟频率)
- 2) 程序总时钟周期数 = 程序总指令条数 * 平均**CPI**
- 3) 按程序指令条数的权重计算:

$$\text{平均CPI} = \sum (F_i * \text{CPI}_i)$$

F_i 为第*i*种指令的所占比例, CPI_i 为第*i*种指令的**CPI**。



华中科技大学

二、计算机系统性能测试

3. 用户**CPU**时间 = 平均**CPI** * 程序总指令条数 * 时钟周期

时钟周期、指令条数、**CPI**三者相互制约

优化用户**CPU**时间时，需要综合考虑。





二、计算机系统性能测试

例：程序**P**在机器**A**上运行需**10 s**， 机器**A**的时钟频率为**400MHz**。

现在要设计一台机器**B**，希望该程序在**B**上运行只需**6 s**。由于机器**B**时钟频率的提高导致了其**CPI**的增加，使得程序**P**在机器**B**上运行时，总时钟周期数是在机器**A**上的**1.2倍**。机器**B**的时钟频率达到**A**的多少倍才能满足设计要求？





二、计算机系统性能测试

解答：用户**CPU时间** = 平均**CPI** * 程序总指令条数 * 时钟周期 = 总时钟周期数 / 时钟频率

$$\text{CPU时间A} = \text{总时钟周期数A} / \text{时钟频率A}$$

$$\text{CPU时间B} = \text{总时钟周期数B} / \text{时钟频率B}$$

总时钟周期数**B** = 1.2 * 总时钟周期数**A**, CPU时间**A** = 10s, CPU时间**B** = 6s

$$\therefore \text{时钟频率B} = \text{总时钟周期数B} / \text{CPU时间B}$$

$$= 1.2 * \text{总时钟周期数A} / \text{CPU时间B}$$

$$= 1.2 * \text{CPU时间A} * \text{时钟频率A} / \text{CPU时间B}$$

$$= 1.2 * 10 * 400\text{M} / 6 = 800\text{M(Hz)}$$





二、计算机系统性能测试

例：机器M上有三类指令A、B、C，其CPI分别为1、2、4。程序P在M上被编译为目标代码P1和P2，P1含A、B、C三类指令条数为8、2、2，P2含A、B、C三类指令条数为2、5、3。

P1和P2，哪个总指令条数少？哪个执行速度快？平均CPI分别为多少？



二、计算机系统性能测试

解答：

	P1	P2
总指令条数	$8+2+2=12$	$2+5+3=10$
总时钟周期数	$8*1+2*2+2*4=20$	$2*1+5*2+3*4=24$
平均CPI	$20/12=1.67$	$24/10=2.4$

∴ P2的总指令条数少，P1的执行速度快。



三、用指令执行速度进行性能评估

早期使用每秒完成指令条数衡量计算机性能。

1. **MIPS**为每秒执行多少百万条记录。

对平均**CPI**求倒数能得到**MIPS**值



三、用指令执行速度进行性能评估

例：机器**M**上有四类指令**A**、**B**、**C**、**D**，其**CPI**分别为1、2、2、2。程序**P**在**M**上编译，优化前的四类代码为43%、21%、12%、24%；优化后**A**类指令条数减少50%，其他指令条数不变。假设机器**M**频率为50MHz。优化前后程序的**CPI**分别为多少？优化前后程序的**MIPS**各是多少？



三、用指令执行速度进行性能评估

解答：

	优化前	优化后
总指令条数	100	$43/2 + 21 + 12 + 24 = 78.5$
总时钟周期数	$43*1+21*2+12*2+24*2=157$	$43/2*1+21*2+12*2+24*2=135.5$
平均CPI	1.57	$135.5/78.5=1.73$
MIPS	$50M/1.57=31.8$	$50M/1.73=28.9$

注：用**MIPS**对不同的机器进行性能比较，有可能不准确或不客观。



三、用指令执行速度进行性能评估

2. 浮点操作速度单位——GFLOPS、TFLOPS、PFLOPS

浮点运算实际上包括了所有涉及小数的运算，在某些应用软件中常常出现，比整数运算更费时间。

现今大部分的处理器中都有浮点运算器。

每秒浮点运算次数所量测的实际上就是浮点运算器的执行速度。

最常用来测量每秒浮点运算次数的基准程序（**benchmark**）之一，就是**Linpack**。





三、用指令执行速度进行性能评估

MFLOPS——每秒一百万 ($=10^6$) 次的浮点运算

GFLOPS——每秒拾亿 ($=10^9$) 次的浮点运算

TFLOPS——每秒万亿 ($=10^{12}$) 次的浮点运算

PFLOPS——每秒千万亿 ($=10^{15}$) 次的浮点运算

EFLOPS——每秒百亿亿 ($=10^{18}$) 次的浮点运算



華科技大學

三、用指令执行速度进行性能评估

2017年6月19号公布全球超级计算机500强：

第一名：中国国家超级计算无锡中心研制的“神威·太湖之光”浮点运算速度为每秒**9.3**亿亿次。

第二名：国防科大研制的“天河二号”超级计算机，每秒**3.386**亿亿次的浮点运算速度，之前曾获得五连冠。





華中科技大學

三、用指令执行速度进行性能评估

Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	National Supercomputing Center in Wuxi China	Sunway TaihuLight - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway NRCP	10,649,600	93,014.6	125,435.9	15,371
2	National Super Computer Center in Guangzhou China	Tianhe-2 (MilkyWay-2) - TH-IVB-FEP Cluster, Intel Xeon E5-2692 12C 2.200GHz, TH Express-2, Intel Xeon Phi 31S1P NUDT	3,120,000	33,862.7	54,902.4	17,808
3	Swiss National Supercomputing Centre (CSCS) Switzerland	Piz Daint - Cray XC50, Xeon E5-2690v3 12C 2.6GHz, Aries interconnect , NVIDIA Tesla P100 Cray Inc.	361,760	19,590.0	25,326.3	2,272





四、用基准程序进行性能评估

1. 用基准程序来评测计算机的性能

- 基准测试程序是专门用来进行性能评价的一组程序
- 基准程序通过运行实际负载来反映计算机的性能
- 最好的基准程序是用户实际使用的程序或典型的简单程序



四、用基准程序进行性能评估

2. 基准程序的缺陷

现象：基准程序的性能与某段短代码密切相关时，会被利用以得到不当的性能评测结果

手段：硬件系统设计人员或编译器开发者针对这些代码片段进行特殊的优化，使得执行这段代码的速度非常快





四、用基准程序进行性能评估

例：Intel Pentium处理器运行SPECint时用了公司内部使用的特殊编译器，使其性能极高

例：矩阵乘法程序SPECmatrix300有99%的时间运行在一行语句上，有些厂商用特殊编译器优化该语句，使性能达VAX11/780的729.8倍！

